

Eugene Moshchyn

Dr. Qi Cheng

CS 4413, Section 1

February 4, 2026

Assignment 1

Exercise 1.2-2 Suppose that for inputs of size n on a particular computer, insertion sort runs in $8n^2$ steps and merge sort runs in $64n \lg n$ steps. For which values of n does insertion sort beat merge sort?

Solution

To compare the efficiency of these two algorithms, their performance functions are plotted in Figure 1. Merge sort is represented by the blue line and insertion sort by the red line.

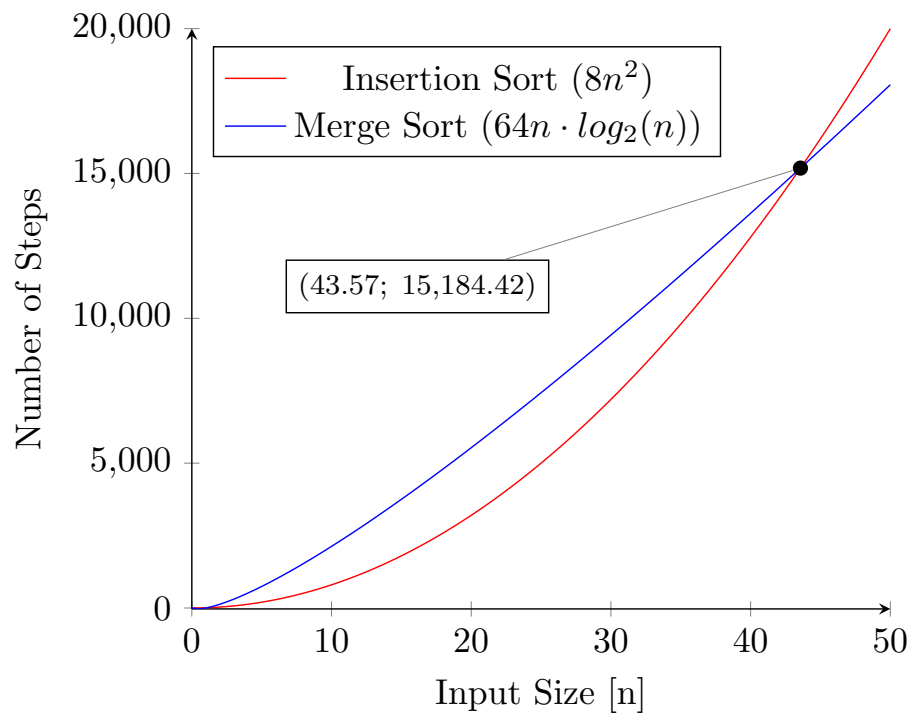


Figure 1: Comparison of Insertion Sort vs. Merge Sort

The graph indicates that for small values of n , insertion sort performs fewer steps than merge sort. To find the exact crossing point, we solve for n where the two functions intersect:

$$\begin{aligned}
 8n^2 &= 64n \cdot \log_2(n) \\
 \Rightarrow \cancel{8n} \cdot n &= \cancel{8n} \cdot 8 \cdot \log_2(n) \\
 \Rightarrow n &= 8 \cdot \log_2(n) \\
 \Rightarrow \frac{n}{8} &= \log_2(n) \\
 \Rightarrow 2^{\frac{n}{8}} &= n
 \end{aligned}$$

Solving this equation for n (either via numerical methods or a calculator) results in an intersection at approximately $n \approx 43.55926$. Therefore, for $2 \leq n \leq 43$, insertion sort is more efficient. For $n \geq 44$, merge sort becomes a better choice.

Exercise 2.1-1 Using Figure 2.2 as a model, illustrate the operation of INSERTION-SORT on an array initially containing the sequence $\langle 31; 41; 59; 26; 41; 58 \rangle$.

Solution

The figure below provides a step-by-step graphical trace of INSERTION-SORT applied to the sequence $\langle 31; 41; 59; 26; 41; 58 \rangle$:

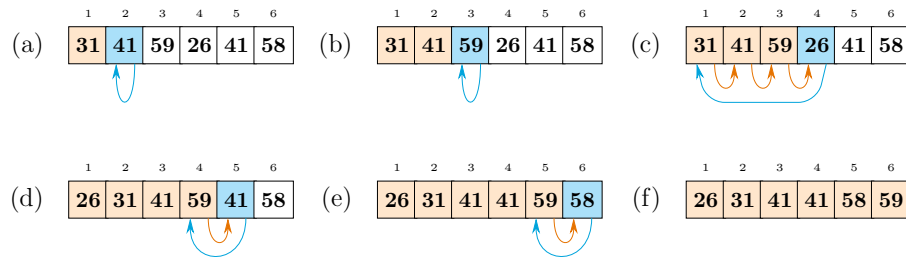


Figure 2: Sorting Steps for sequece $\langle 31; 41; 59; 26; 41; 58 \rangle$ using INSERTION-SORT

Exercise 2.1-3 Rewrite the INSERTION-SORT procedure to sort into monotonically decreasing instead of monotonically increasing order.

Solution

The original INSERTION-SORT procedure from the textbook (Chapter 2, Section 1, page 19) is as follows:

```

INSERTION-SORT( $A, n$ )
1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 

```

The comparison logic governing the sort order occurs on line 5. The condition $A[j] > key$ results in an ascending (increasing) order because it shifts elements to the right only if they are greater than the key. To sort in monotonically decreasing order, we reverse the comparison operator to $A[j] < key$.

The modified code with the change highlighted in red is:

```

INSERTION-SORT( $A, n$ )
1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] < key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 

```

Exercise 2.1-5 Consider the problem of adding two n -bit binary integers a and b , stored in two n -element arrays $A[0 : n - 1]$ and $B[0 : n - 1]$, where each element is either 0 or 1, $a = \sum_{i=0}^{n-1} A[i] \cdot 2^i$, and $b = \sum_{i=0}^{n-1} B[i] \cdot 2^i$. The sum $c = a + b$ of the two integers should be stored in binary form in an $(n + 1)$ -element array $C[0 : n]$, where $c = \sum_{i=0}^n C[i] \cdot 2^i$. Write a procedure ADD-BINARY-INTEGERS that takes as input arrays A and B , along with the length n , and returns array C holding the sum.

Solution

To determine the algorithm's behavior, consider the addition of two binary sequences, A and B , of length n :

$$\begin{array}{ll}
 \text{(a)} & A = \langle 0, 1, 1, 0, 1 \rangle, \quad n = 5 \\
 & B = \langle 1, 0, 0, 1, 0 \rangle, \quad n = 5 \\
 \hline
 & \text{SumBinaryLists}(A, B, n) \implies C = \langle 1, 1, 1, 1, 1, 0 \rangle, \quad n = 6
 \end{array}$$

$$\begin{array}{rcl}
 \text{(b)} & A = \langle 1, 0, 0, 0, 1 \rangle, & n = 5 \\
 & B = \langle 1, 1, 0, 1, 0 \rangle, & n = 5 \\
 \hline
 \text{SumBinaryLists}(A, B, n) \implies & C = \langle 0, 0, 1, 1, 1, 0 \rangle, & n = 6
 \end{array}$$

$$\begin{array}{rcl}
 \text{(c)} & A = \langle 0, 0, 0, 0, 0 \rangle, & n = 5 \\
 & B = \langle 0, 0, 0, 0, 0 \rangle, & n = 5 \\
 \hline
 \text{SumBinaryLists}(A, B, n) \implies & C = \langle 0, 0, 0, 0, 0, 0 \rangle, & n = 6
 \end{array}$$

$$\begin{array}{rcl}
 \text{(d)} & A = \langle 1, 1, 1, 1, 1 \rangle, & n = 5 \\
 & B = \langle 1, 1, 1, 1, 1 \rangle, & n = 5 \\
 \hline
 \text{SumBinaryLists}(A, B, n) \implies & C = \langle 0, 1, 1, 1, 1, 1 \rangle, & n = 6
 \end{array}$$

Based on the examples above and the exercise specifications, we can conclude that a carry mechanism is required. The array C requires an extra element (the (n) -th position) to store the final overflowing value.

Here is the algorithm:

```

SUMBINARYLISTS( $A, B, n$ )
1   $C[0:n] = \{0\}$ 
2   $carry = 0$ 
3  for  $i = 1$  to  $n - 1$ 
4       $sum = A[i] + B[i] + carry$ 
5       $C[i] = sum \bmod 2$ 
6       $carry = \lfloor sum/2 \rfloor$ 
7   $C[n] = carry$ 
8  return  $C$ 

```